

Principles of Programming Languages

Lecture 3: Semantics

Andrei Arusoaie¹

¹Department of Computer Science

October 19, 2016

Outline

Semantics: introduction

The K “machinery”

IMP: a simple imperative language in K

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
3
```

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

4

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
```

3

Semantics: motivation

C

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

Java

```
-$ cat File.java
public class File {
    ... void main(...) {
        int x = 0;
        println((x=1) + (x=2));
    }
}
-$ javac File.java
-$ java File
3
```

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 8.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

4

GCC: LLVM version 8.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
```

3

Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 8.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```


Semantics: motivation

GCC: 5.4.0-6 ubuntu

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
4
```

GCC: LLVM version 8.0.0 - clang

```
-$ cat test.c
int main()
{
    int x;
    return (x=1) + (x=2);
}
-$ gcc test.c
-$ ./a.out ; echo $?
3
```

Demo

► out-of-lifetime.c

Semantics

- ▶ Semantics is concerned with the **meaning** of language constructs
- ▶ Semantics must be **unambiguous**
- ▶ Semantics must be **flexible**

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic - informal

Informal semantics (examples): **natural language**

Rationale for the ANSI C Programming Language:

- ▶ “Trust the programmer”
- ▶ “Don’t prevent the programmer from doing what needs to be done”
- ▶ “Keep the language small and simple”
- ▶ “Provide only one way to do an operation”
- ▶ “Make it fast, even if it is not guaranteed to be portable”

Inexact! – could lead to **undefined behavior** in programs

Semantic styles

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: K semantics, Small-step SOS, Big-Step SOS

Semantic styles

Some (formal) semantics styles:

- ▶ operational
- ▶ denotational
- ▶ axiomatic

We will focus more on **operational** semantics styles: K semantics, Small-step SOS, Big-Step SOS

A framework for defining PL semantics

A framework for defining semantics needs to be:

- ▶ expressive
- ▶ modular
- ▶ executable
- ▶ based on some "logic of programs" - enables reasoning

The K “machinery”

- ▶ We saw that K can be used to define **syntax**
- ▶ For **semantics** we have to understand the following key ingredients:
 - ▶ Computations
 - ▶ Configurations
 - ▶ Rules

Computations

- ▶ **Computation = unit of calculus**
- ▶ When processing language constructs we organise them as sequences of such units (think about executing statements in order)
- ▶ Using such sequences imposes an evaluation order
- ▶ Examples:
 - ▶ $2 + 3$
 - ▶ $x = 2;$
 - ▶ K sequence: $x = 2; \curvearrowright y = 3;$

Computations

- ▶ Computation = unit of calculus
- ▶ When processing language constructs we organise them as sequences of such units (think about executing statements in order)
- ▶ Using such sequences imposes an evaluation order
- ▶ Examples:
 - ▶ $2 + 3$
 - ▶ $x = 2;$
 - ▶ K sequence: $x = 2; \curvearrowright y = 3;$

Computations

- ▶ Computation = unit of calculus
- ▶ When processing language constructs we organise them as sequences of such units (think about executing statements in order)
- ▶ Using such sequences imposes an evaluation order
- ▶ Examples:
 - ▶ $2 + 3$
 - ▶ $x = 2;$
 - ▶ K sequence: $x = 2; \curvearrowright y = 3;$

Computations

- ▶ Computation = unit of calculus
- ▶ When processing language constructs we organise them as sequences of such units (think about executing statements in order)
- ▶ Using such sequences imposes an evaluation order
- ▶ Examples:
 - ▶ $2 + 3$
 - ▶ $x = 2;$
 - ▶ K sequence: $x = 2; \curvearrowright y = 3;$

Configurations

- ▶ We need a way to model program states
- ▶ K configurations:
 - ▶ nested structures of **cells**: `<k> x = 3; </k>`
 - ▶ Example:

```
configuration <T>
    <k> x = 3; <k>
    <env> x |-> 0 <env>
</T>
```

- ▶ The `<k>` cell is special: it contains the \$PGM as a list of computations
- ▶ Example: `<k> x = 3; ↻ y = 7; <k>`

Configurations

- ▶ We need a way to model program states
- ▶ K configurations:
 - ▶ nested structures of **cells**: `<k> x = 3; </k>`
 - ▶ Example:

```
configuration <T>  
    <k> x = 3; <k>  
    <env> x |-> 0 <env>  
</T>
```

- ▶ The `<k>` cell is special: it contains the \$PGM as a list of computations
- ▶ Example: `<k> x = 3; ↻ y = 7; <k>`

Configurations

- ▶ We need a way to model program states
- ▶ K configurations:
 - ▶ nested structures of **cells**: `<k> x = 3; </k>`
 - ▶ Example:

```
configuration <T>  
    <k> x = 3; <k>  
    <env> x |-> 0 <env>  
</T>
```

- ▶ The `<k>` cell is special: it contains the \$PGM as a list of computations
- ▶ Example: `<k> x = 3; ↻ y = 7; <k>`

Rules

- ▶ K **rules** establish transitions between configurations
- ▶ Example:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

Rules

- ▶ K **rules** establish transitions between configurations
- ▶ Example:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

Rules at work

The K rule:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

An example:

```
<T>
  <k> x = 2; ↷ y = 2; <k>
  <env> x |-> 0 <env>
</T>
=>
<T>
  <k> y = 2; <k>
  <env> x |-> 2 <env>
</T>
```


Rules at work

The K rule:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

An example:

```
<T>
  <k> x = 2; ↷ y = 2; <k>
  <env> x |-> 0 <env>
</T>
=>
<T>
  <k> y = 2; <k>
  <env> x |-> 2 <env>
</T>
```

Rules at work

The K rule:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

An example:

```
<T>
  <k> x = 2; ↷ y = 2; <k>
  <env> x |-> 0 <env>
</T>
=>
<T>
  <k> y = 2; <k>
  <env> x |-> 2 <env>
</T>
```

Abstractions

Configuration abstraction: write in rules only what is *changing*!

The K rule:

```
rule <T>  
  <k> X = V; ↷ K </k>  
  <env> X |-> _ </env>  
</T>  
=>  
<T>  
  <k> K </k>  
  <env> X |-> V </env>  
</T>
```

An example:

```
<T>  
  <k> x = 2; ↷ y = 2; <k>  
  <env> x |-> 0 <env>  
</T>  
=>  
<T>  
  <k> y = 2; <k>  
  <env> x |-> 2 <env>  
</T>
```

Abstractions

Configuration abstraction: write in rules only what is *changing*!

The K rule:

```
rule <T>  
  <k> X = V; ↷ K </k>  
  <env> X |-> _ </env>  
</T>  
=>  
<T>  
  <k> K </k>  
  <env> X |-> V </env>  
</T>
```

An example:

```
<T>  
  <k> x = 2; ↷ y = 2; <k>  
  <env> x |-> 0 <env>  
</T>  
=>  
<T>  
  <k> y = 2; <k>  
  <env> x |-> 2 <env>  
</T>
```

Abstractions

Configuration abstraction: write in rules only what is *changing*!

The K rule:

```
rule <T>
  <k> X = V; ↷ K </k>
  <env> X |-> _ </env>
</T>
=>
<T>
  <k> K </k>
  <env> X |-> V </env>
</T>
```

An example:

```
<T>
  <k> x = 2; ↷ y = 2; <k>
  <env> x |-> 0 <env>
</T>
=>
<T>
  <k> y = 2; <k>
  <env> x |-> 2 <env>
</T>
```

Abstractions

Configuration abstraction: write in rules only what is *changing*!

The K rule:

An example:

rule

```
<k> X = V; ↷ K </k>  
<env> X |-> _ </env>
```

=>

```
<k> K </k>  
<env> X |-> V </env>
```

```
<k> x = 2; ↷ y = 2; <k>  
<env> x |-> 0 <env>
```

=>

```
<k> y = 2; <k>  
<env> x |-> 2 <env>
```

The K definition compiler fills the context for us!

Abstractions

Configuration abstraction: write in rules only what is *changing*!

The K rule:

An example:

rule

```
<k> X = V; ↷ K </k>  
<env> X |-> _ </env>
```

=>

```
<k> K </k>  
<env> X |-> V </env>
```

```
<k> x = 2; ↷ y = 2; <k>  
<env> x |-> 0 <env>
```

=>

```
<k> y = 2; <k>  
<env> x |-> 2 <env>
```

The K definition compiler fills the context for us!

Abstractions

Local rewrites: *put the rewrite inside the cell!*

```
rule
  <k> X = V;  $\hookrightarrow$  K </k>
  <env> X |-> _ </env>
=>
  <k> K </k>
  <env> X |-> V </env>
```

```
rule
  <k> (X = V; => .)  $\hookrightarrow$  K </k>
  <env> X |-> (_ => V) </env>
```


Abstractions

Local rewrites: *put the rewrite inside the cell!*

rule

<k> X = V; $\hookrightarrow$ K </k>

<env> X $\mapsto$ _ </env>

=>

<k> K </k>

<env> X $\mapsto$ V </env>

rule

<k> (X = V; => .) $\hookrightarrow$ K </k>

<env> X $\mapsto$ (_ => V) </env>

Abstractions

Local rewrites: *put the rewrite inside the cell!*

rule

<k> X = V; $\hookrightarrow$ K </k>

<env> X $\mapsto$ _ </env>

=>

<k> K </k>

<env> X $\mapsto$ V </env>

rule

<k> (X = V; => .) $\hookrightarrow$ K </k>

<env> X $\mapsto$ (_ => V) </env>

Abstractions

Local rewrites: *put the rewrite inside the cell!*

rule

<k> X = V; $\hookrightarrow$ K </k>

<env> X $\mapsto$ _ </env>

=>

<k> K </k>

<env> X $\mapsto$ V </env>

rule

<k> (X = V; => .) ... </k>

<env> X $\mapsto$ (_ => V) </env>

Heating and cooling rules

- Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V)</env>
```

- This rule works fine when V is a **result**!
- Example: $x = 2 + 2$;
- If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- Heating and cooling rules:
 - syntax `Stmt ::= Id "=" Exp ";" [strict(2)]`
 - Heating: $2 + 2 \curvearrowright x = \square$;
 - Compute result: $4 \curvearrowright x = \square$;
 - Cooling: $x = 4$;
 - Now we can apply the rule!

Heating and cooling rules

- Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V) </env>
```

- This rule works fine when V is a **result**!
- Example: $x = 2 + 2$;
- If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- Heating and cooling rules:
 - syntax `Stmt ::= Id "=" Exp ";" [strict(2)]`
 - Heating: $2 + 2 \curvearrowright x = \square$;
 - Compute result: $4 \curvearrowright x = \square$;
 - Cooling: $x = 4$;
 - Now we can apply the rule!

Heating and cooling rules

- ▶ Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V)</env>
```

- ▶ This rule works fine when V is a **result**!
- ▶ Example: $x = 2 + 2$;
- ▶ If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- ▶ Heating and cooling rules:

- ▶ syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
- ▶ Heating: $2 + 2 \leadsto x = \square$;
- ▶ Compute result: $4 \leadsto x = \square$;
- ▶ Cooling: $x = 4$;
- ▶ Now we can apply the rule!

Heating and cooling rules

- ▶ Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V)</env>
```

- ▶ This rule works fine when V is a **result**!
- ▶ Example: $x = 2 + 2$;
- ▶ If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- ▶ **Heating and cooling rules:**

- ▶ syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
- ▶ Heating: $2 + 2 \leadsto x = \square$;
- ▶ Compute result: $4 \leadsto x = \square$;
- ▶ Cooling: $x = 4$;
- ▶ Now we can apply the rule!

Heating and cooling rules

- Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V)</env>
```

- This rule works fine when V is a **result**!
- Example: $x = 2 + 2$;
- If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- **Heating and cooling rules:**

- syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
- Heating: $2 + 2 \curvearrowright x = \square$;
- Compute result: $4 \curvearrowright x = \square$;
- Cooling: $x = 4$;
- Now we can apply the rule!

Heating and cooling rules

- ▶ Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> (_ => V)</env>
```

- ▶ This rule works fine when V is a **result**!
- ▶ Example: $x = 2 + 2$;
- ▶ If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- ▶ **Heating and cooling rules:**

- ▶ syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
- ▶ Heating: $2 + 2 \hookrightarrow x = \square$;
- ▶ Compute result: $4 \hookrightarrow x = \square$;
- ▶ Cooling: $x = 4$;
- ▶ Now we can apply the rule!

Heating and cooling rules

- ▶ Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> ( _ => V)</env>
```

- ▶ This rule works fine when V is a **result**!
- ▶ Example: $x = 2 + 2$;
- ▶ If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- ▶ **Heating and cooling rules:**
 - ▶ syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
 - ▶ Heating: $2 + 2 \curvearrowright x = \square$;
 - ▶ Compute result: $4 \curvearrowright x = \square$;
 - ▶ Cooling: $x = 4$;
 - ▶ Now we can apply the rule!

Heating and cooling rules

- Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> (_ => V)</env>
```

- This rule works fine when V is a **result**!
- Example: $x = 2 + 2$;
- If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- **Heating and cooling rules:**
 - syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
 - Heating: $2 + 2 \curvearrowright x = \square$;
 - Compute result: $4 \curvearrowright x = \square$;
 - Cooling: $x = 4$;
 - Now we can apply the rule!

Heating and cooling rules

- Recall:

```
rule
  <k> (X = V; => .) ...</k>
  <env> X |-> (_ => V)</env>
```

- This rule works fine when V is a **result**!
- Example: $x = 2 + 2$;
- If V is not a value (e.g., $2 + 2$) then we *evaluate* it! How?
- **Heating and cooling rules:**
 - syntax $\text{Stmt} ::= \text{Id} \text{ "=" } \text{Exp} \text{ ";" } [\text{strict}(2)]$
 - Heating: $2 + 2 \curvearrowright x = \square$;
 - Compute result: $4 \curvearrowright x = \square$;
 - Cooling: $x = 4$;
 - Now we can apply the rule!

IMP

- ▶ We will define a simple imperative language in K
- ▶ Features:
 - ▶ Arithmetic and boolean expressions
 - ▶ Statements: assignments, decisional statement, loops, blocks, sequences
- ▶ DEMO

IMP

- ▶ We will define a simple imperative language in K
- ▶ Features:
 - ▶ Arithmetic and boolean expressions
 - ▶ Statements: assignments, decisional statement, loops, blocks, sequences
- ▶ DEMO

Lab - this week

- ▶ Extend IMP with various features (2 p)